

SYLVAIN FRARESSO, GAËL JEAN-ALBERT,
TITOUAN MARTINEAU, THOMAS SAUREL

Transformation de Fourier discrète et applications à la compression des sons et des images

1^{er} mai 2026

Table des matières

1	Introduction	2
2	Les signaux	2
2.1	Définition	2
2.2	Echantillonnage	3
3	Transformée de Fourier	5
3.1	Transformée de Fourier Continue	5
3.2	Transformée de Fourier Discrète	6
3.3	Spectre d'une image	7
4	Transformée de Fourier rapide	7
4.1	Algorithme de Cooley-Tukey	7
4.2	Exemple de calcul	8
4.3	Complexité algorithmique	9
4.4	Approche itérative	9
4.5	Optimisations diverses	10
5	La compression JPEG	12
5.1	Transformée en Cosinus Discrète (DCT)	12
5.2	Transformation en blocs	15
5.3	Compression du spectre	16
6	Métrique d'évaluation d'algorithmes de compression d'images	17
6.1	Erreur Moyenne Quadratique	17
6.2	Peak Signal-to-Noise Ratio	18
6.3	Structural Similarity Index Measure	19

1 Introduction

Au début du XIX^{ème}, le mathématicien Joseph Fourier travaille à la résolution de l'équation de la chaleur [8], une équation aux dérivées partielles modélisant le phénomène de conduction thermique. Dans le cadre de ses travaux, il introduit la notion de séries de Fourier, un développement en série trigonométrique des fonctions périodiques, et plus généralement la transformée de Fourier. Ces deux concepts seront alors étudiés minutieusement et donneront lieu à d'importantes découvertes en mathématiques, physique et informatique.

Le présent document aura pour objectif de formaliser la transformée de Fourier dans le cadre de la théorie du traitement du signal afin de comprendre comment cet outil peut être exploité en informatique pour la compression d'images numériques et de signaux sonores. On s'attardera en particulier sur l'implémentation et les applications de l'algorithme FFT (« Fast Fourier Transform » pour transformée de Fourier rapide) permettant de calculer très efficacement des transformées de Fourier.

FIGURE 1 – Joseph Fourier (1768-1830).

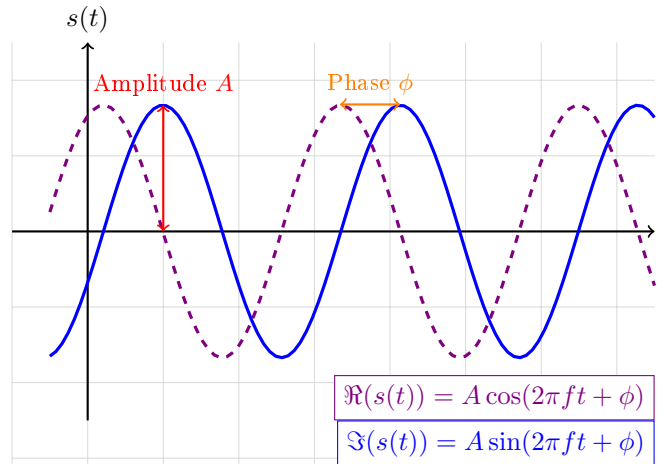


2 Les signaux

2.1 Définition

Commençons par introduire la notion de signal. On appelle **signal analogique** ou **continu**, une fonction de $\mathbb{R}$, représentant le temps, vers $\mathbb{C}$ [11]. Bien que les mesures physiques soient réelles, un signal complexe permet de conserver la **phase** au travers de son argument et son **amplitude** au travers de son module.

FIGURE 2 – Parties réelle et imaginaire d'un signal complexe[5]



Le signal $s(t) = Ae^{i(2\pi ft + \phi)}$ représenté ici est un signal dit "pur". En réalité, la plupart des signaux ont des formes beaucoup plus complexes et ne ressemblent pas à une simple sinusoïde.

Le traitement numérique du signal nécessite une discrétisation pour y appliquer des algorithmes informatiques.

2.2 Echantillonnage

Le passage au numérique demande de ne conserver que certaines valeurs isolées du signal. Nous devons alors introduire la notion de **distribution de Dirac**. Dans le cadre de notre étude, nous utiliserons une définition alternative [4], plus faible que la définition usuelle du Dirac. Cette définition plus intuitive est parfaitement suffisante pour l'utilisation que l'on en fera et ne fait pas appel à la théorie de la mesure.

Soit $(f_\alpha)_{\alpha \in \mathbb{R}^+}$ la famille de fonctions définies par :

$$f_\alpha : t \mapsto \begin{cases} \frac{1}{2\alpha} & \text{si } t \in [-\alpha; \alpha] \\ 0 & \text{sinon} \end{cases}$$

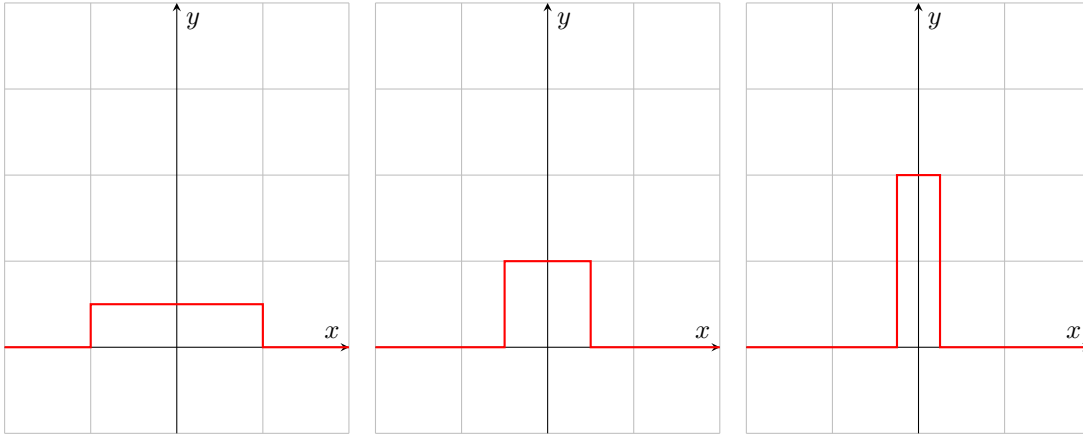
On définit l'action du Dirac δ sur une fonction g , continue en 0, par :

$$\int_{-\infty}^{+\infty} g(t)\delta(t)dt := \lim_{\alpha \rightarrow 0} \int_{-\infty}^{+\infty} g(t)f_\alpha(t)dt$$

Soit $\alpha > 0$,

$$\int_{-\infty}^{+\infty} f_\alpha(t)dt = \int_{-\alpha}^{\alpha} \frac{1}{2\alpha} dt = \frac{1}{2\alpha} \int_{-\alpha}^{\alpha} dt = 1$$

FIGURE 3 – f_α pour $\alpha = 1, \frac{1}{2}, \frac{1}{4}$



En appliquant la définition de l'action du Dirac sur la fonction constante $g = 1$, on obtient :

$$\int_{-\infty}^{+\infty} \delta(t)dt = \lim_{\alpha \rightarrow 0^+} \int_{-\infty}^{+\infty} f_\alpha(t)dt = \lim_{\alpha \rightarrow 0^+} 1 = 1$$

En traitement du signal, on considère souvent abusivement que δ est une fonction réelle, valant 0 partout sauf en 0 où elle vaut « $+\infty$ », mais il s'agit bien d'une distribution caractérisée par :

Sa concentration : $\forall t \neq 0, \delta(t) = 0$

Sa masse : $\int_{-\infty}^{+\infty} \delta(t)dt = 1$

On fera également attention à garder à l'esprit que la notation sous forme d'intégrale impropre sur $\mathbb{R}$ faisant intervenir le Dirac dans l'intégrande fait toujours référence à la définition de l'action du Dirac sous forme de limite et ne pose aucun problème de convergence du moment que g est intégrable au voisinage de 0.

L'intérêt, pour le traitement du signal, de la distribution de Dirac est sa capacité à isoler une valeur ponctuelle d'un signal. En effet, si on définit de la même manière, l'action du Dirac $\delta(t - t_0)$, centré en t_0 , par l'identité $\int_{-\infty}^{+\infty} g(t)\delta(t - t_0)dt = \lim_{\alpha \rightarrow 0+} \int_{-\infty}^{+\infty} g(t)f_\alpha(t - t_0)dt$, alors, on obtient, pour toute fonction g continue en t_0 , la relation suivante :

$$\int_{-\infty}^{+\infty} g(t)\delta(t - t_0)dt = g(t_0)$$

Démonstration. Soit g une fonction continue en $t_0 \in \mathbb{R}$, on a :

$$\int_{-\infty}^{+\infty} g(t)\delta(t - t_0)dt = \lim_{\alpha \rightarrow 0+} \int_{-\infty}^{+\infty} g(t)f_\alpha(t - t_0)dt$$

Soit $\alpha > 0$,

$$\begin{aligned} \int_{-\infty}^{+\infty} g(t)f_\alpha(t - t_0)dt &= \int_{t_0-\alpha}^{t_0+\alpha} \frac{g(t)}{2\alpha} dt \\ &= \frac{1}{2\alpha} \int_{t_0-\alpha}^{t_0+\alpha} g(t)dt \\ &= \frac{1}{2\alpha} \left(\int_{t_0-\alpha}^{t_0+\alpha} g(t_0)dt + \int_{t_0-\alpha}^{t_0+\alpha} (g(t) - g(t_0)) dt \right) \\ &= g(t_0) + \frac{1}{2\alpha} \int_{t_0-\alpha}^{t_0+\alpha} (g(t) - g(t_0)) dt \end{aligned}$$

Or,

$$\begin{aligned} \left| \frac{1}{2\alpha} \int_{t_0-\alpha}^{t_0+\alpha} (g(t) - g(t_0)) dt \right| &\leq \frac{1}{2\alpha} \int_{t_0-\alpha}^{t_0+\alpha} |g(t) - g(t_0)| dt \\ &\leq \sup_{t \in [t_0-\alpha; t_0+\alpha]} |g(t) - g(t_0)| \rightarrow 0 \quad \text{par continuité de } g \text{ en } t_0 \end{aligned}$$

Ainsi, on a bien $\lim_{\alpha \rightarrow 0+} \int_{-\infty}^{+\infty} g(t)f_\alpha(t - t_0)dt = g(t_0)$.

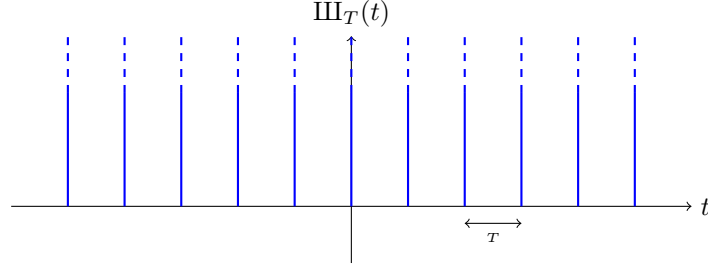
□

Remarque. Cette relation s'apparente au produit de convolution $g * \delta$.

Pour obtenir un échantillonnage d'un signal, il faut réaliser plusieurs extractions de valeurs à l'aide du Dirac. Il est alors pertinent d'introduire le **peigne de Dirac** de période T , noté III_T :

$$\text{III}_T(t) = \sum_{n \in \mathbb{Z}} \delta(t - nT)$$

FIGURE 4 – Peigne de Dirac de période T

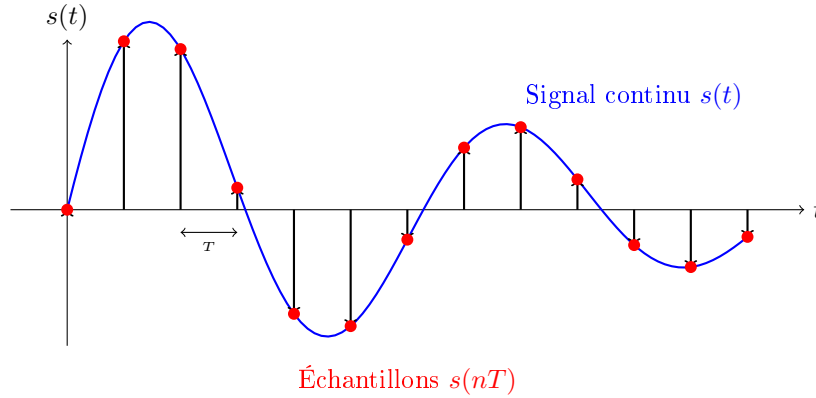


Le signal échantillonné $s_e(t)$ est alors modélisé par le produit du signal analogique $s(t)$ et du peigne de Dirac III_T :

$$s_e(t) = s(t) \cdot \text{III}_T(t) = \sum_{n \in \mathbb{Z}} s(nT) \delta(t - nT)$$

Cette expression transforme une fonction continue en une suite de coefficients $s(nT)$ qui pourront être traités avec des méthodes numériques.

FIGURE 5 – Échantillonnage d'une fonction continue



Pour qu'un signal soit traitable par un algorithme informatique, il faut qu'il comporte un nombre fini de valeurs. Il suffit alors de considérer un signal continu sur un intervalle de longueur fini puis de l'échantillonner pour ne garder qu'un nombre fini N de valeurs. On peut alors considérer ce signal échantillonné fini comme un vecteur :

$$x = (s(0), s(T), s(2T), \dots, s((N-1)T)) = (x_0, x_1, \dots, x_{N-1}) \in \mathbb{C}^N$$

3 Transformée de Fourier

3.1 Transformée de Fourier Continue

Le mathématicien Joseph Fourier a démontré que la plupart des signaux que l'on souhaite étudier dans le cadre du traitement de signal peut s'écrire comme une somme d'exponentielles complexes [13]. Ainsi, un signal est caractérisé par sa phase, son amplitude mais surtout par sa, ou ses, fréquences (largeur des oscillations). L'enjeu de cette section va être d'introduire des outils permettant de passer

du domaine temporel, où le signal est caractérisé par le temps t , vers le domaine fréquentiel, paramétré par des fréquences.

Avant d'aborder la transformée de Fourier discrète, il est nécessaire de définir un outil qui permet d'extraire la fréquence d'un signal. Pour une fonction f représentant un signal quelconque, on utilise la définition suivante :

Soit f intégrable sur $\mathbb{R}$. La **transformée de Fourier** de f , notée $TF[f]$, est la fonction :

$$TF[f] : x \mapsto \int_{-\infty}^{+\infty} f(t) \cdot e^{-2i\pi xt} dt$$

Remarque. Dans le cadre du traitement de signal, la fonction à laquelle on applique une TF est un signal, $x(t)$, sa transformée de Fourier est alors notée $X(f)$ où f désigne la fréquence. On a :

$$\begin{aligned} X(f) &= \int_{-\infty}^{+\infty} x(t) \cdot e^{-i2\pi ft} dt \\ X(\omega) &= \int_{-\infty}^{+\infty} x(t) \cdot e^{-i\omega t} dt \quad \text{où } \omega = 2\pi f, \text{ la pulsation} \end{aligned}$$

3.2 Transformée de Fourier Discrète

Soit $x = (x_0, x_1, \dots, x_{N-1}) \in \mathbb{C}^N$ le vecteur du signal échantillonné. On appelle **Transformée de Fourier Discrète** [12] de x , le vecteur $X = (X_0, X_1, \dots, X_{N-1}) \in \mathbb{C}^N$ avec :

$$X_k = \sum_{n=0}^{N-1} x_n e^{-i\frac{2\pi}{N} kn} \quad \forall k \in \{0, \dots, N-1\}$$

Cette formule permet de passer du domaine temporel (x_k) au domaine fréquentiel (X_k).

De manière analogue, on peut définir la Transformée de Fourier Discrète Inverse qui permet de revenir du domaine fréquentiel au domaine temporel :

$$x_k = \frac{1}{N} \sum_{n=0}^{N-1} X_n e^{i\frac{2\pi}{N} kn} \quad \forall k \in \{0, \dots, N-1\}$$

On a alors $TFDI(TFD(x_n)) = x_n$

Démonstration. Soit $(x_0, x_1, \dots, x_{N-1}) \in \mathbb{C}^N$. Soit $0 \leq n < N$.

$$\begin{aligned} TFDI(TFD(x_n)) &= \frac{1}{N} \sum_{k=0}^{N-1} \left(\sum_{m=0}^{N-1} x_m e^{-i\frac{2\pi}{N} km} \right) e^{i\frac{2\pi}{N} kn} \\ &= \frac{1}{N} \sum_{m=0}^{N-1} x_m \left(\sum_{k=0}^{N-1} e^{-i\frac{2\pi}{N} k(m-n)} \right) \end{aligned}$$

Posons :

$$S_N := \sum_{k=0}^{N-1} e^{-i\frac{2\pi}{N} k(m-n)}$$

On remarque que S_N est une somme partielle d'une suite géométrique de raison $q = e^{-i\frac{2\pi}{N}(m-n)}$.

$$\begin{aligned}
\text{Si } n = m, \quad q = e^0 = 1 &\implies S_N = \sum_{k=0}^{N-1} 1 = N \\
\text{Si } n \neq m, \quad S_N = \frac{1 - q^N}{1 - q} \text{ or, } q^N = e^{-i2\pi(m-n)} = 1 &\text{ car } m - n \in \mathbb{Z} \text{ donc } S_N = 0 \\
&\implies TFDI(TFD(x_n)) = \frac{1}{N} \sum_{m=0}^{N-1} x_m \cdot N\delta_{nm} = \frac{1}{N} x_n \cdot N = x_n
\end{aligned}$$

□

3.3 Spectre d'une image

4 Transformée de Fourier rapide

La FFT (*Fast Fourier Transform*, transformée de Fourier rapide) désigne une famille d'algorithmes permettant de calculer « rapidement » la transformée de Fourier discrète d'un signal. En particulier, la complexité temporelle des algorithmes FFT est en $\Theta(n \log(n))$, ces algorithmes sont donc bien plus efficace que l'algorithme naïf consistant à appliquer directement la définition de la TFD dont la complexité est en $\Theta(n^2)$.

Les premières ébauches de FFT remontent au début du XIX^{ème}, Gauss en avait alors imaginé le principe[2] mais le concept a réellement connu un regain d'intérêt dans la deuxième moitié du XX^{ème} siècle avec l'essor de l'informatique.

4.1 Algorithme de Cooley-Tukey

En 1965, les mathématiciens James Cooley et John Tukey publient un article présentant un algorithme de FFT appelé algorithme de Cooley-Tukey[1] qui relance l'intérêt pour la transformée de Fourier rapide. Il s'agit dans sa version la plus simple d'un algorithme récursif.

Dans un premier temps, il faut reformuler quelque peu les termes du problèmes[6]. On a vu précédemment que la TFD d'un signal $x = (x_0, \dots, x_{N-1})$ était le vecteur $X = (X_0, \dots, X_{N-1})$ avec $X_k = \sum_{n=0}^{N-1} x_n e^{-i\frac{2\pi}{N}kn}$. Or, si on pose $P = \sum_{n=0}^{N-1} x_n X^n$ et $\omega_N = e^{-i\frac{2\pi}{N}}$ une racine primitive N -ième de l'unité, alors, calculer X_k revient à évaluer P en $\omega_N^k = e^{-i\frac{2\pi}{N}k}$. On suppose dans la suite que N est une puissance de 2.

On peut alors exploiter cette reformulation pour réduire le coût de calcul, on va décomposer P en deux polynômes de degré inférieur. En particulier, on a,

$$\begin{aligned}
P(X) &= \sum_{n=0}^{N-1} x_n X^n \\
&= \sum_{n=0}^{\frac{N}{2}-1} x_{2n} X^{2n} + \sum_{n=0}^{\frac{N}{2}-1} x_{2n+1} X^{2n+1} \\
&= \sum_{n=0}^{\frac{N}{2}-1} x_{2n} (X^2)^n + X \sum_{n=0}^{\frac{N}{2}-1} x_{2n+1} (X^2)^n
\end{aligned}$$

On pose alors,

$$P_{\text{pair}}(X) = \sum_{n=0}^{\frac{N}{2}-1} x_{2n} X^n \quad \text{et} \quad P_{\text{impair}}(X) = \sum_{n=0}^{\frac{N}{2}-1} x_{2n+1} X^n$$

on obtient finalement,

$$P(X) = P_{\text{pair}}(X^2) + X P_{\text{impair}}(X^2)$$

Ainsi, pour $k \in \{0, \dots, N-1\}$,

$$X_k = P(\omega_N^k) = P_{\text{pair}}(\omega_N^{2k}) + \omega_N^k P_{\text{impair}}(\omega_N^{2k}).$$

Or, $\omega_N^2 = e^{-i\frac{4\pi}{N}} = e^{-i\frac{2\pi}{N/2}} = \omega_{N/2}$ et P_{pair} et P_{impair} sont de degré $\frac{N}{2}$. Ainsi, les termes $P_{\text{pair}}(\omega_N^{2k})$ et $P_{\text{impair}}(\omega_N^{2k})$ correspondent à des transformées de Fourier discrètes de taille $\frac{N}{2}$.

On obtient donc les relations suivantes, pour $k \in \{0, \dots, \frac{N}{2} - 1\}$:

$$X_k = E_k + \omega_N^k O_k, \quad X_{k+\frac{N}{2}} = E_k - \omega_N^k O_k$$

où

$$E_k = P_{\text{pair}}(\omega_N^{2k}), \quad O_k = P_{\text{impair}}(\omega_N^{2k})$$

Cette décomposition permet de calculer une TFD de taille N à partir de deux TFD de taille $\frac{N}{2}$. On

observe en outre que si $N = 1$, on a $X_0 = \sum_{n=0}^0 x_0 e^{-i\frac{2\pi}{N} \cdot 0 \cdot n} = x_0$. On en déduit l'algorithme récursif suivant :

Algorithm 1: Algorithme de Cooley-Tukey[6]

Données: *Un signal $x = (x_0, \dots, x_{N-1})$ et la racine de l'unité $\omega = e^{-i\frac{2\pi}{N}}$.*

Résultat: *La transformée de Fourier discrète du signal x , $X = (X_0, \dots, X_{N-1})$.*

Fonction $\text{fft}(x, \omega)$:

```

si  $N = 1$  alors
  retourner  $(x_0)$ 
sinon
   $E \leftarrow \text{fft}((x_0, x_2, \dots, x_{N-2}), \omega^2)$ 
   $O \leftarrow \text{fft}((x_1, x_3, \dots, x_{N-1}), \omega^2)$ 
   $\alpha \leftarrow 1$ 
  pour  $k = 0$  à  $\frac{N}{2} - 1$  faire
     $X_k \leftarrow E[k] + \alpha \times O[k]$ 
     $X_{k+\frac{N}{2}} \leftarrow E[k] - \alpha \times O[k]$ 
     $\alpha \leftarrow \alpha \times \omega$ 
  retourner  $(X_0, \dots, X_{N-1})$ 

```

Notons que la racine de l'unité ω est donnée en entrée de la fonction **fft**, on évite ainsi de la recalculer en fonction de N à chaque appel récursif. En pratique une fonction *wrapper* (enveloppante), ne prenant pas ω en paramètre, se chargera de calculer ω_N avant d'appeler **fft** une première fois.

4.2 Exemple de calcul

Lorsqu'on atteint l'étape de combinaison ("le papillon"), on considère les FFT des sous-signaux pairs (E) et impairs (O) de taille $n = 4$ déjà calculées. De même avant la première boucle on a $\alpha = 1$.

Itération 1 : $k = 0$

- **Calcul des composantes :**
 - $X_0 = E[0] + \alpha \times O[0] = E[0] + 1 \times O[0]$
 - $X_4 = E[0] - \alpha \times O[0] = E[0] - 1 \times O[0]$
- **Mise à jour du facteur tournant :**
 - $\alpha \leftarrow \alpha \times \omega = 1 \times \omega = \omega$

Itération 2 : $k = 1$

- **Calcul des composantes :**
 - $X_1 = E[1] + \alpha \times O[1] = E[1] + \omega \times O[1]$
 - $X_5 = E[1] - \alpha \times O[1] = E[1] - \omega \times O[1]$
- **Mise à jour du facteur tournant :**
 - $\alpha \leftarrow \alpha \times \omega = \omega \times \omega = \omega^2$

Le processus continue ainsi pour $k = 2$ et $k = 3$ afin de compléter les 8 éléments du signal final X .

Étape	Calculs des sorties X	Mise à jour de α
$k = 2$	$X_2 = E[2] + \omega^2 \cdot 0[2], \quad X_6 = E[2] - \omega^2 \cdot 0[2]$	$\alpha \leftarrow \omega^2 \cdot \omega$
$k = 3$	$X_3 = E[3] + \omega^3 \cdot 0[3], \quad X_7 = E[3] - \omega^3 \cdot 0[3]$	$\alpha \leftarrow \omega^3 \cdot \omega$

On reconstruit $X = (X_0, X_1, X_2, X_3, X_4, X_5, X_6, X_7)$.

4.3 Complexité algorithmique

L'algorithme de Cooley-Tukey a une complexité algorithmique en $\Theta(n \log(n))$ où n est la taille du signal à traiter. Ce résultat se démontre à l'aide du *master theorem*[10] (aussi appelé « théorème sur les récurrences par partition »).

En effet, si on note $\tau(n)$ le nombre d'opérations élémentaires pour exécuter **fft** sur un signal de longueur n , on a,

$$\begin{aligned} \tau(n) &= 2\tau\left(\frac{n}{2}\right) + \Theta(n) \\ &= a\tau\left(\frac{n}{b}\right) + f(n) \quad \text{avec } a = b = 2 \text{ et } f(n) = \Theta(n) \end{aligned}$$

On calcule alors,

$$n^{\log_b a} = n^{\log_2 2} = n^1 = n$$

On obtient,

$$f(n) = \Theta(n) = \Theta(n^{\log_b a})$$

On en déduit donc, d'après le *master theorem*, que $\tau(n) = \Theta(n^{\log_b a} \log(n)) = \Theta(n \log(n))$.

4.4 Approche itérative

Bien que l'implémentation fidèle de l'algorithme de Cooley-Tukey décrit précédemment se révèle bien plus efficace que l'algorithme naïf, issu de la définition de la TFD, il reste très lent comparé à l'implémentation de référence pour python fournie dans le module numpy.

Il est néanmoins possible d'améliorer notre implémentation en abandonnant l'approche récursive au profit d'une approche itérative[3]. Cela nous permettra de supprimer les copies et allocations de tableaux et éviter les coûts supplémentaires, en temps et en mémoire, liés à la récursion.

L'idée centrale consiste à simuler les appels récursifs « du bas vers le haut ». Le problème, pour accomplir cela, est que l'on ne sait pas, à priori, comment recomposer les sous-tableaux en partant de l'étape avec N sous-tableaux de taille 1. En réalité, le découpage des sous-tableaux en indices pairs/impairs à chaque étape de la récursion permet de « lire », dans l'écriture binaire des indices des éléments, leur parcours dans les différents sous-tableaux.

Plus précisément, si le tableau à traiter a pour longueur 2^m , on peut représenter l'indice k d'un élément x en binaire $k = \overline{b_{m-1}b_{m-2} \dots b_0}^2$. Alors, lors du premier appel, x est placé dans le sous-tableau pair ou impair en fonction de la parité de b_0 . Lors du deuxième appel, tous les éléments placés avec x ont la même valeur pour b_0 , ainsi séparer éléments pairs et impairs se fait en regardant le bit b_1 . Ce processus se poursuit jusqu'à obtenir des sous-tableaux de taille 1. Ainsi, en partant des sous-tableaux de taille 1, les recompositions à faire se déduisent de la suite $(b_0, b_1, \dots, b_{m-1})$.

À partir de ce constat, on peut obtenir la version itérative de l'algorithme. On commence par réorganiser le tableau en entrée en inversant l'ordre des bits des indices. Une fois le tableau réordonné, on applique les opérations papillons de manière itérative. Une boucle, faisant $\log_2(N)$ tours, remplace les appels récursifs. À l'étape j , on combine des blocs de taille 2^j . On en déduit l'algorithme itératif suivant :

Algorithm 2: Algorithme de Cooley-Tukey itératif

Données: *Un signal* $x = (x_0, \dots, x_{N-1})$.

Résultat: *La transformée de Fourier discrète du signal* $X = (X_0, \dots, X_{N-1})$.

Fonction `fft_iterative(x)` :

```

     $x \leftarrow \text{bit\_reversal\_permutation}(x)$ 
     $t \leftarrow 1$ 
     $m \leftarrow 2$ 
    pour  $j = 1$  à  $\log_2 N$  faire
         $\omega_m \leftarrow e^{\frac{-2i\pi}{m}}$ 
         $\alpha \leftarrow 1$ 
        pour  $l = 0$  à  $t - 1$  faire
            pour  $k = l$  à  $N - 1$  avec un pas de  $m$  faire
                 $u \leftarrow x_k$ 
                 $v \leftarrow \alpha \times x_{k+t}$ 
                 $x_k \leftarrow u + v$ 
                 $x_{k+t} \leftarrow u - v$ 
             $\alpha \leftarrow \alpha \times \omega_m$ 
         $t \leftarrow m$ 
         $m \leftarrow 2 \times m$ 
    retourner  $(x_0, \dots, x_{N-1})$ 

```

4.5 Optimisations diverses

Bien que la réécriture de l'algorithme en itératif présente des avantages évidents comme le gain de mémoire, une rapide comparaison avec notre première version récursive montre que le gain d'efficacité est marginal.

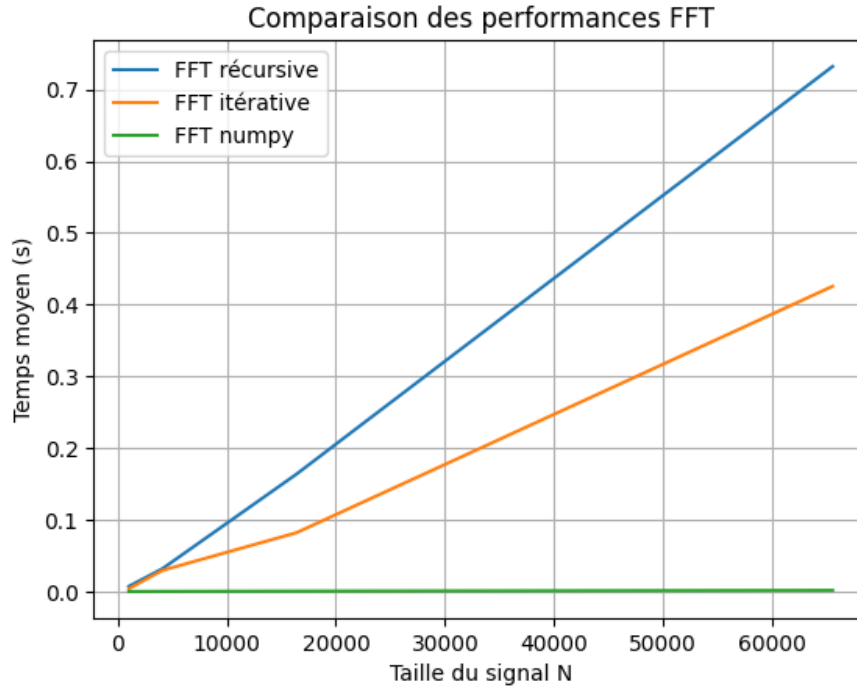


FIGURE 6 – Temps de calcul moyen en fonction de la taille du signal pour différentes FFT.

On peut observer sur ce graphique que notre version itérative est plus rapide en moyenne que la version récursive de la FFT mais elles restent toutes les deux très loin de l'implémentation de référence de numpy. Cet écart est dû, dans une moindre mesure, à des optimisations supplémentaires mis en oeuvre dans l'implémentation numpy, mais surtout au fait que la librairie numpy fait appel à du code C compilé là où notre implémentation en pur python est simplement interprétée.

Pour palier ce problème, une solution simple serait de réécrire tout notre code en C, néanmoins, en python toujours, il est possible de faire un compromis consistant en la compilation à la volée (*just in time compilation*[7]). En temps normal, l'interpréteur python traduit dans un premier temps le code python en bytecode, qui sera ensuite exécuté par une machine virtuel. En utilisant une librairie de compilation à la volée, par exemple numba, le bytecode produit est compilé, s'il est jugé que cela apporterait un gain de performance, en langage machine.

Avec cette amélioration, nos deux implémentations de Cooley-Tukey se rapprochent des performances de l'implémentation de numpy. On peut en outre remarquer que l'implémentation itérative semble plus bénéficier de la compilation à la volée.

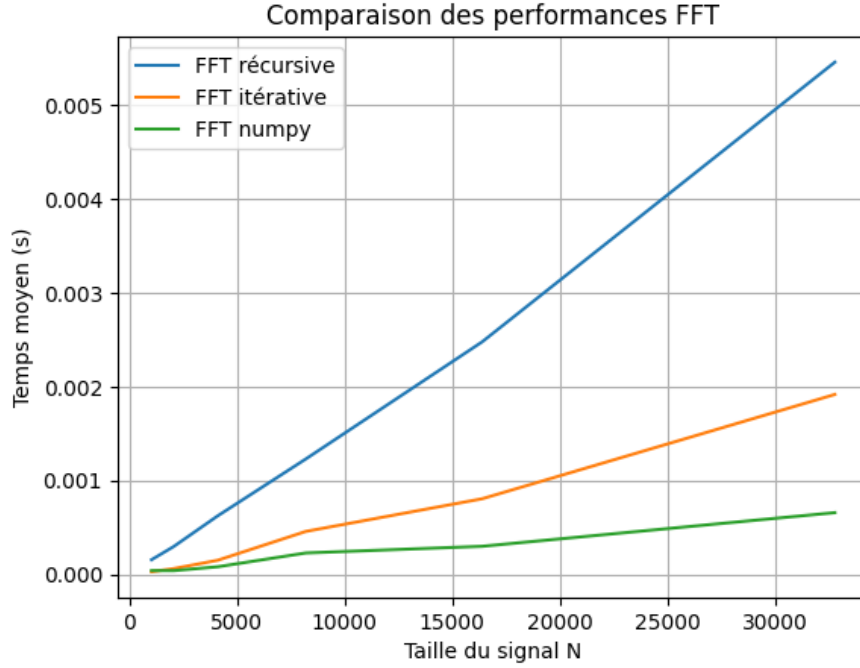


FIGURE 7 – Temps de calcul moyen en fonction de la taille du signal pour différentes FFT, avec compilation à la volée.

5 La compression JPEG

Le format **JPEG (Joint Photographic Experts Group)** est un format d'image très utilisé. Contrairement aux formats d'image tels que PNG ou BMP qui sont des formats sans perte, JPEG utilise une compression avec perte similaire à la transformée de Fourier discrète appelée **Transformée en cosinus discrète (DCT)**[9]. La compression JPEG repose sur la décomposition de l'image en blocs carrés. Chaque bloc est décomposé en une somme de cosinus de fréquences différentes. Le spectre ainsi obtenu peut alors être compressé en ne gardant que les fréquences les plus faibles. Il ne reste alors qu'à faire la transformation inverse (analogue à la transformée de Fourier inverse) et de rassembler les blocs pour obtenir une image compressée.

5.1 Transformée en Cosinus Discrète (DCT)

La **Transformée en Cosinus Discrète (DCT)** est une transformation linéaire orthogonale qui projette un vecteur réel $x = (x_0, x_1, \dots, x_{N-1}) \in \mathbb{R}^N$ sur une base de fonctions cosinus. Elle est largement utilisée en traitement du signal et en compression d'images.

La DCT-II[14], qui est la plus couramment utilisée, est définie par :

$$X_k = \sum_{n=0}^{N-1} x_n \cos \left[\frac{\pi}{N} \left(n + \frac{1}{2} \right) k \right], \quad k = 0, 1, \dots, N-1$$

Pour obtenir une transformation orthonormée, on ajoute des facteurs de normalisation :

$$X_k = \alpha_k \sum_{n=0}^{N-1} x_n \cos \left[\frac{\pi}{N} \left(n + \frac{1}{2} \right) k \right]$$

avec

$$\alpha_k = \begin{cases} \sqrt{\frac{1}{N}} & \text{si } k = 0 \\ \sqrt{\frac{2}{N}} & \text{si } k \geq 1 \end{cases}$$

On définit les fonctions de base de la DCT :

$$e_k(n) = \alpha_k \cos\left(\frac{\pi}{N}\left(n + \frac{1}{2}\right)k\right), \quad k = 0, \dots, N-1$$

Les (e_k) forment une base orthonormale pour le produit scalaire usuel sur $\mathbb{R}^N$.

Démonstration. On considère le produit scalaire usuel sur $\mathbb{R}^N$: $\langle x, y \rangle = \sum_{n=0}^{N-1} x_n y_n$. On calcule :

$$\langle e_k, e_l \rangle = \alpha_k \alpha_l \sum_{n=0}^{N-1} \cos\left(\frac{\pi}{N}\left(n + \frac{1}{2}\right)k\right) \cos\left(\frac{\pi}{N}\left(n + \frac{1}{2}\right)l\right)$$

On remarque alors que $\sum_{n=0}^{N-1} \cos\left(\theta\left(n + \frac{1}{2}\right)\right) = \frac{\sin(N\theta/2)}{2\sin(\theta)}$, d'où :

$$\begin{aligned} \langle e_k, e_l \rangle &= \frac{\alpha_k \alpha_l}{2} \left(\frac{\sin(N(\theta_k - \theta_l))}{2\sin((\theta_k - \theta_l)/2)} + \frac{\sin(N(\theta_k + \theta_l))}{2\sin((\theta_k + \theta_l)/2)} \right) \\ &= \frac{\alpha_k \alpha_l}{4} \left(\frac{\sin(\pi(k-l))}{\sin(\pi(k-l)/2N)} + \frac{\sin(\pi(k+l))}{\sin(\pi(k+l)/2N)} \right) \end{aligned}$$

— Si $k \neq l$, alors $\langle e_k, e_l \rangle = \sin(\pi(k-l)) = \sin(\pi(k+l)) = 0$

— Si $k = l \neq 0$, le premier terme vaut $2N$ et le second est nul : $\langle e_k, e_k \rangle = \frac{\alpha_k^2}{4} 2N = \frac{1}{2N} 2N = 1$

— Si $k = l = 0$, les deux termes valent $2N$ par continuité en 0 : $\langle e_0, e_0 \rangle = \frac{\alpha_0^2}{4} (2N + 2N) = 1$

Ainsi, la famille (e_k) forme une base orthonormale de $\mathbb{R}^N$. □

La DCT peut être vue comme une décomposition du signal sur une base orthogonale de cosinus de fréquences croissantes. L'avantage des cosinus comparés aux exponentielles complexes de la transformée de Fourier réside dans le fait que les valeurs obtenues restent réelles.

La DCT possède plusieurs propriétés importantes :

- **Linéarité** : la transformation est linéaire.
- **Orthogonalité** : les vecteurs sont orthogonaux.
- **Énergie concentrée** : l'énergie du signal est concentrée dans un petit nombre de coefficients.
- **Inverse** : il existe une transformée inverse (IDCT) permettant de reconstruire exactement le signal initial

L'orthogonalité est un atout important de cette transformation car elle permet de conserver l'énergie

d'un signal. En effet, si on note $E = \sum_{n=0}^{N-1} x_n^2 = \|x\|^2$ l'énergie du signal $x = (x_0, \dots, x_{N-1})$ et A la

matrice orthogonale associée à cette transformation alors :

$$\begin{aligned}
\|X\|^2 &= \langle Ax, Ax \rangle \\
&= Ax(Ax)^t \quad (\text{en identifiant } \mathcal{M}_{1,1}(\mathbb{R}) \text{ à } \mathbb{R}) \\
&= (Ax)^t Ax \\
&= x^t (A^t A) x \\
&= x^t x \\
&= \|x\|^2
\end{aligned}$$

Cette propriété assure donc qu'il n'y a pas de perte d'information après la transformation. Ainsi, on peut appliquer une transformation inverse afin de retrouver le signal original. La transformée inverse, notée IDCT est :

$$x_n = \sum_{k=1}^{N-1} \alpha_k X_k \cos \left[\frac{\pi}{N} \left(n + \frac{1}{2} \right) k \right], \quad k = 0, 1, \dots, N-1$$

On a alors $\text{IDCT}(\text{DCT}(x_n)) = x_n$

Démonstration. On a, par définition,

$$\begin{aligned}
X_k &= \alpha_k \sum_{m=0}^{N-1} x_m \cos \left[\frac{\pi}{N} \left(m + \frac{1}{2} \right) k \right] \\
x_n &= \sum_{k=0}^{N-1} \alpha_k X_k \cos \left[\frac{\pi}{N} \left(n + \frac{1}{2} \right) k \right]
\end{aligned}$$

On développe,

$$\begin{aligned}
\text{IDCT}(\text{DCT}(x_n)) &= \sum_{k=0}^{N-1} \alpha_k \left(\alpha_k \sum_{m=0}^{N-1} x_m \cos \left[\frac{\pi}{N} \left(m + \frac{1}{2} \right) k \right] \right) \cos \left[\frac{\pi}{N} \left(n + \frac{1}{2} \right) k \right] \\
&= \sum_{k=0}^{N-1} \left(\alpha_k^2 \sum_{m=0}^{N-1} x_m \cos \left[\frac{\pi}{N} \left(m + \frac{1}{2} \right) k \right] \right) \cos \left[\frac{\pi}{N} \left(n + \frac{1}{2} \right) k \right] \\
&= \sum_{m=0}^{N-1} x_m \left(\sum_{k=0}^{N-1} \alpha_k^2 \cos \left[\frac{\pi}{N} \left(m + \frac{1}{2} \right) k \right] \cos \left[\frac{\pi}{N} \left(n + \frac{1}{2} \right) k \right] \right) \\
&= \sum_{m=0}^{N-1} x_m \left(\sum_{k=0}^{N-1} \left(\alpha_k \cos \left[\frac{\pi}{N} \left(m + \frac{1}{2} \right) k \right] \right) \left(\alpha_k \cos \left[\frac{\pi}{N} \left(n + \frac{1}{2} \right) k \right] \right) \right)
\end{aligned}$$

Posons $e_i(k) = \alpha_k \cos \left[\frac{\pi}{N} \left(i + \frac{1}{2} \right) k \right]$, $e_i = (e_i(0), e_i(1), \dots, e_i(N-1)) \in \mathbb{R}^N$, alors,

$$\begin{aligned}
\text{IDCT}(\text{DCT}(x_n)) &= \sum_{m=0}^{N-1} x_m \left(\sum_{k=0}^{N-1} e_m(k) e_n(k) \right) \\
&= \sum_{m=0}^{N-1} x_m \langle e_m, e_n \rangle = \sum_{m=0}^{N-1} x_m \delta_{m,n} = x_n
\end{aligned}$$

□

5.2 Transformation en blocs

Le format JPEG divise les images en blocs de pixels (de taille 8×8 dans notre implémentation conformément au standard JPEG). On peut alors appliquer la DCT sur chacun des blocs. Cette découpe est avantageuse car les pixels au sein d'un même bloc sont souvent corrélés, en effet, les images représentent souvent des objets au sens large et des pixels voisins sont donc susceptibles d'appartenir à un même objet. De plus, il existe des implémentations efficaces permettant d'éviter des calculs répétitifs. On peut alors calculer les coefficients en cosinus une seule fois :

$$(C_1, C_2, C_3, C_4, C_5, C_6, C_7) = \sqrt{\frac{2}{N}} \cdot \left(\cos \frac{\pi}{16}, \cos \frac{2\pi}{16}, \cos \frac{3\pi}{16}, \cos \frac{4\pi}{16}, \cos \frac{5\pi}{16}, \cos \frac{6\pi}{16}, \cos \frac{7\pi}{16} \right)$$

Une optimisation possible est l'utilisation de la propriété de symétrie du cosinus. Commençons par poser,

$$\theta_{n,k} := \frac{\pi}{8} \left(n + \frac{1}{2} \right) k$$

Remarquons que

$$\theta_{7-n,k} = \frac{\pi}{8} \left(7 - n + \frac{1}{2} \right) k = \frac{\pi}{8} \left(\frac{15}{2} - n \right) k$$

Ainsi,

$$x_{7-n} \cos(\theta_{7-n,k}) = x_{7-n} (-1)^k \cos(\theta_{n,k}) = \pi k - \theta_{n,k}$$

On peut alors utiliser cette propriété pour réduire le nombre de calculs à effectuer.

$$\begin{aligned} X_k &= \alpha_k \sum_{n=0}^7 [x_n \cos(\theta_{n,k})] \\ &= \alpha_k \sum_{n=0}^3 [x_n \cos(\theta_{n,k}) + x_{7-n} \cos(\theta_{7-n,k})] \\ &= \alpha_k \sum_{n=0}^3 [x_n + (-1)^k x_{7-n}] \cos(\theta_{n,k}) \end{aligned}$$

On obtient alors les X_k à l'aide des produits matriciels suivants :

$$\begin{pmatrix} X_0 \\ X_2 \\ X_4 \\ X_6 \end{pmatrix} = \begin{pmatrix} C_4 & C_4 & C_4 & C_4 \\ C_2 & C_6 & -C_6 & -C_2 \\ C_4 & -C_4 & -C_4 & C_4 \\ C_6 & -C_2 & C_2 & -C_6 \end{pmatrix} \begin{pmatrix} x_0 + x_7 \\ x_1 + x_6 \\ x_2 + x_5 \\ x_3 + x_4 \end{pmatrix}$$

$$\begin{pmatrix} X_1 \\ X_3 \\ X_5 \\ X_7 \end{pmatrix} = \begin{pmatrix} C_1 & C_3 & C_5 & C_7 \\ C_3 & -C_7 & -C_1 & -C_5 \\ C_5 & -C_1 & C_7 & C_3 \\ C_7 & -C_5 & C_3 & -C_1 \end{pmatrix} \begin{pmatrix} x_0 - x_7 \\ x_1 - x_6 \\ x_2 - x_5 \\ x_3 - x_4 \end{pmatrix}$$

où l'ordre des C_i sur la ligne correspondant à X_k est donné par

$$i \equiv k(2n+1) \pmod{16} \quad \text{avec } n \text{ l'indice de colonne}$$

Cette relation provient de l'identification $C_i = \cos\left(\frac{i\pi}{16}\right) = \cos\left(\frac{\pi}{8}\left(n + \frac{1}{2}\right)k\right)$, ce qui implique finalement $i \equiv k(2n+1) \pmod{16}$.

5.3 Compression du spectre

Une fois dans le domaine fréquentiel, on peut compresser l'image en ne gardant qu'une certaine portion des fréquences les plus basses dans chaque bloc du spectre. On observe que les basses fréquences sont positionnées dans le coin supérieur gauche de chacun des blocs de l'image.

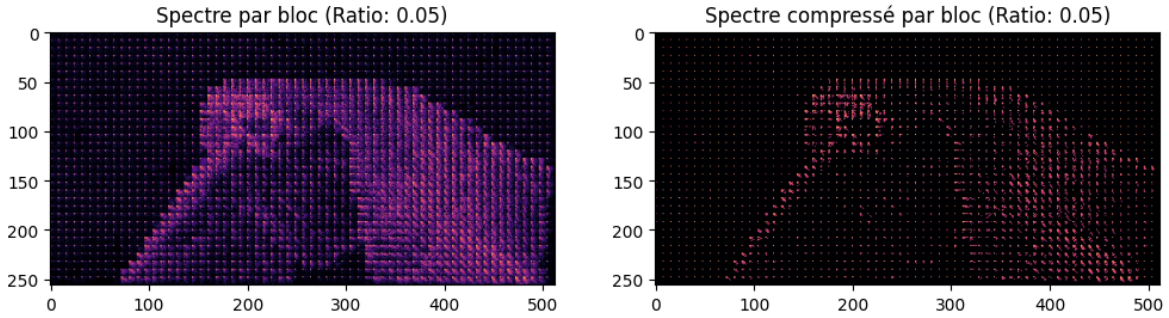


FIGURE 8 – Spectre de l'image après application de la transformée en cosinus discrète

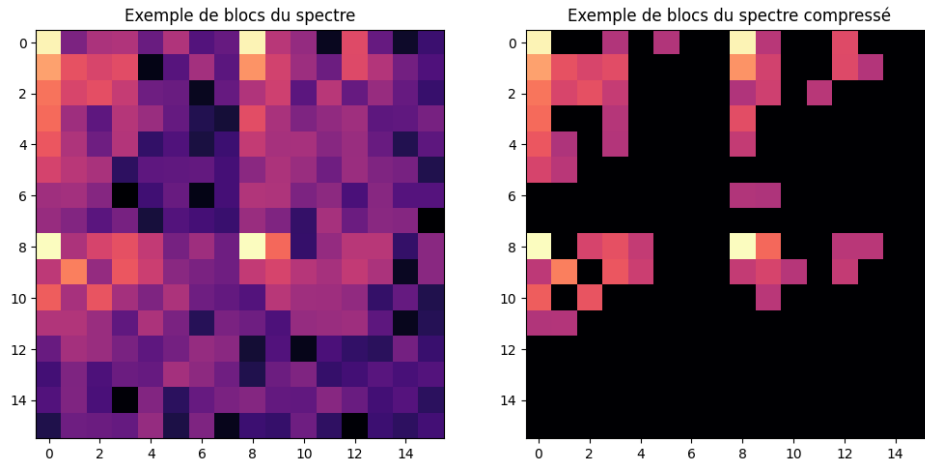


FIGURE 9 – Exemple de 4 blocs (8×8) contenus dans le spectre

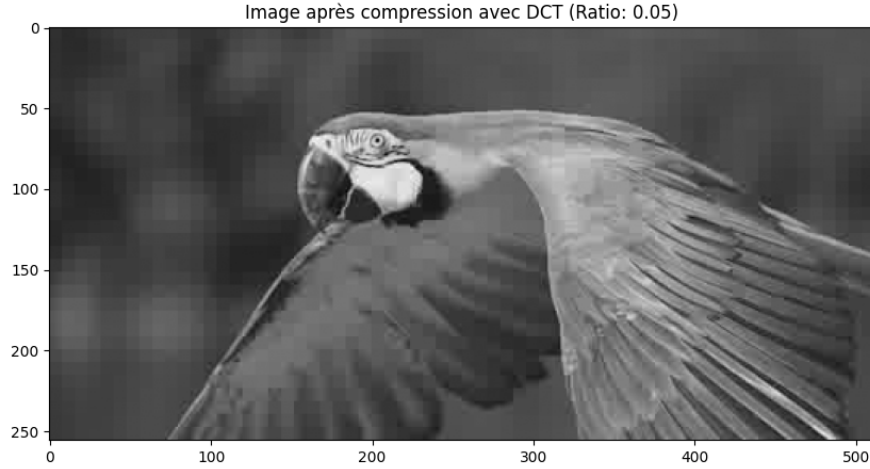


FIGURE 10 – Image finale après application de la DCT et de la compression avec un taux de conservation d'information de 5%

On observe que le fond est plus compressé, d'où les blocs plus visibles, car l'image d'origine comprend un fond peu détaillé. L'objet contenant le plus de détails, le perroquet, est peu affecté par la compression.

6 Métrique d'évaluation d'algorithmes de compression d'images

Nous avons vu plusieurs algorithmes pour la compression des images. Il devient alors nécessaire d'introduire différentes métriques pour quantifier les différences entre l'image originale et celles compressées par les algorithmes.

Nous verrons dans cette partie différentes métriques, certaines pour quantifier les différences purement mathématiques et d'autres pour approximer les différences perceptibles par l'oeil humain. Ces métriques seront utiles afin de déterminer plus tard le taux de compression optimal pour faire un compromis entre la taille du fichier et la qualité de l'image.

6.1 Erreur Moyenne Quadratique

Une première métrique permettant de quantifier la compression est l'**erreur moyenne quadratique (MSE)**. Cette métrique est très utilisée, notamment dans le domaine de l'apprentissage automatique. Elle mesure la moyenne des distances carrées entre la valeur initiale et la nouvelle valeur.

$$MSE = \frac{1}{3MN} \sum_{c \in \{r, g, b\}} \sum_{i=0}^{M-1} \sum_{j=0}^{N-1} (I_O^c(i, j) - I_C^c(i, j))^2$$

où M et N représentent respectivement la hauteur et la largeur de l'image, $I_O^c(i, j)$ (resp. $I_C^c(i, j)$) est l'intensité du pixel (i, j) sur le canal de la couleur c de l'image originale (resp. compressée). Cette métrique se distingue de l'erreur moyenne absolue en accentuant les grands écarts de par sa nature quadratique. Cela permet de détecter l'apparition d'artéfacts très visibles sur l'image.

Une erreur moyenne quadratique faible indique que chaque pixel sur chaque canal de couleur de l'image compressée a une valeur proche de celle du pixel correspondant de l'image originale.

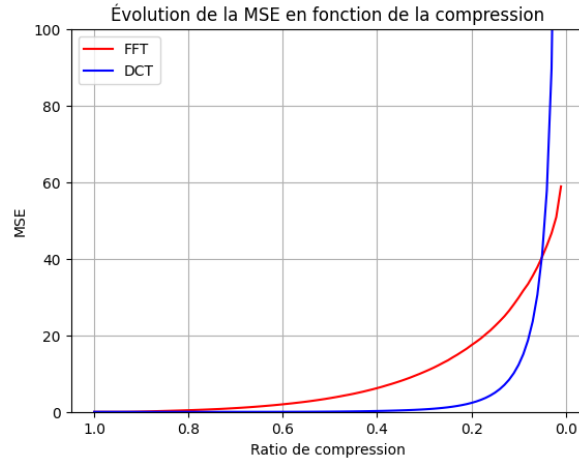


FIGURE 11 – Evolution de la MSE en fonction du ratio de la compression

On observe que la MSE sur l'image compressée avec la DCT est inférieure à la MSE de l'image compressée avec la FFT.

Cette métrique telle quelle est peu utilisée en traitement du signal mais sert souvent à calculer une autre métrique plus adaptée pour la quantification de la compression de signal.

6.2 Peak Signal-to-Noise Ratio

La métrique **Peak Signal-to-Noise Ratio (PSNR)** est la plus utilisée pour la compression d'images. Elle est basée sur le calcul de l'erreur moyenne quadratique. Le PSNR s'exprime en décibels dB.

$$PSNR = 10 \cdot \log_{10}\left(\frac{255^2}{MSE}\right)$$

Cette métrique donne le rapport entre la valeur maximale d'un pixel au carré et la MSE. Le passage au $\log_{10}$ et l'ajout du facteur 10 permettent de réduire l'échelle des valeurs prises par le PSNR afin que cela soit plus facilement interprétable par une personne. Ainsi, une valeur très faible indique une perte conséquente de qualité d'image, alors qu'une valeur élevée indique une image compressée fidèle à l'image originale. En pratique, on rajoute un petit facteur à la MSE (10^{-100} dans notre implémentation) pour éviter une division par 0 si l'on veut calculer le PSNR avec une compression nulle, l'image a donc une *MSE* nulle.

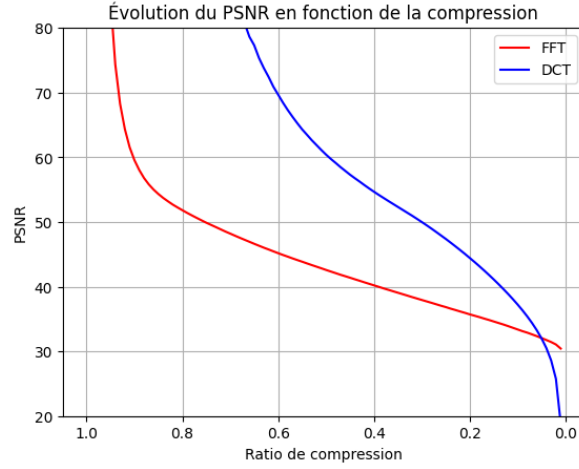


FIGURE 12 – Evolution du PSNR en fonction du ratio de la compression

On observe logiquement que le PSNR est meilleur avec la DCT qu'avec la FFT car la MSE donnée après compression avec DCT était inférieure à la MSE après compression avec FFT.

Les deux métriques vues précédemment étant purement mathématiques, elles restent difficiles à interpréter. De plus, une valeur d'une métrique peut avoir plusieurs interprétations. Ainsi, une modification uniforme peut engendrer la même erreur que l'ajout d'un artéfact concentré en une région de l'image. Il peut alors devenir intéressant de développer de nouvelles métriques plus proches de la vision humaine.

6.3 Structural Similarity Index Measure

La **Structural Similarity Index Measure (SSIM)** se concentre sur la perception de l'oeil humain plutôt que sur la théorie mathématique. Contrairement à la MSE et au PSNR qui repose sur des différences pixel à pixel, la SSIM compare la structure globale de l'image. Elle est calculée à l'aide de deux fenêtres de taille $N \times N$, la première (x) sur l'image originale et la seconde (y) sur l'image compressée :

$$SSIM(x, y) = \frac{(2\mu_x\mu_y + c_1)(2\sigma_x\sigma_y + c_2)(\sigma_{xy} + c_3)}{(\mu_x^2 + \mu_y^2 + c_1)(\sigma_x^2 + \sigma_y^2 + c_2)(\sigma_x\sigma_y + c_3)}$$

avec :

- μ_x : moyenne de x μ_y : moyenne de y σ_x^2 : variance de x σ_y^2 : variance de y
- σ_{xy} : covariance de x et y
- $c_1 = (0.01 \cdot 255)^2 = 6.5025$ $c_2 = (0.03 \cdot 255)^2 = 58.5225$ $c_3 = \frac{c_2}{2} = 29.26125$

Les constantes c_1, c_2 et c_3 ont été établies en fonction des caractéristiques de la vision humaine. Cette formule est construite en combinant trois rapports : la **luminance**, le **contraste** et la **structure**.

La **luminance** compare la luminosité moyenne entre les deux fenêtres :

$$l(x, y) = \frac{2\mu_x\mu_y + c_1}{\mu_x^2 + \mu_y^2 + c_1}$$

Ainsi, plus les moyennes sont proches, plus la luminance tend vers 1. Dans le cas contraire, plus les moyennes s'éloignent, plus la luminance diminue vers 0.

Le **contraste** compare l'amplitude des variations de lumière sur les deux fenêtres :

$$c(x, y) = \frac{2\sigma_x\sigma_y + c_2}{\sigma_x^2 + \sigma_y^2 + c_2}$$

Plus les variances sont proches, plus le contraste tend vers 1. Dans le cas contraire, plus les variances s'éloignent, plus le contraste diminue vers 0.

La **structure** compare la corrélation entre les deux fenêtres :

$$s(x, y) = \frac{cov_{xy} + c_3}{\sigma_x \sigma_y + c_3}$$

Si on omet la constante c_3 , on reconnaît ici le coefficient de corrélation de Pearson. Il mesure la direction d'une relation linéaire entre deux variables. Plus le coefficient de corrélation est proche de 1, plus les variables suivent une même trajectoire.

Finalement, on obtient :

$$SSIM(x, y) = l(x, y) \cdot c(x, y) \cdot s(x, y)$$

Ainsi, plus la luminance, le contraste et la structure sont semblables entre les deux images, plus chaque rapport tend vers 1. On en déduit que plus la SSIM est proche de 1 et plus les images sont semblables entre elles.

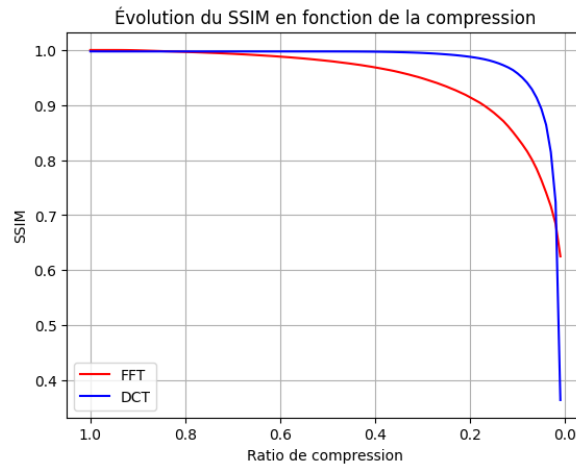


FIGURE 13 – Evolution du SSIM en fonction du ratio de la compression

On observe que la SSIM est bien meilleure avec la DCT qu'avec la FFT. On peut expliquer cela car la DCT regroupe les zones sans détails en carrés et garde les zones détaillées nettes. Ainsi, les zones plus détaillées sont peu modifiées alors que le fond de l'image qui est moins détaillé devient flou au fil de la compression, modifiant peu la perception finale de l'image.

Références

- [1] James W. COOLEY et John W. TUKEY. « An Algorithm for the Machine Calculation of Complex Fourier Series ». In : *Mathematics of Computation* 19.90 (1965), p. 297-301. DOI : 10.1090/S0025-5718-1965-0178586-1.
- [2] M. HEIDEMAN, D. JOHNSON et C. BURRUS. « Gauss and the history of the fast fourier transform ». In : *IEEE ASSP Magazine* 1.4 (1984), p. 14-21. DOI : 10.1109/MASSP.1984.1162257.
- [3] KLAFYVEL. *Jouons à implémenter une transformée de Fourier rapide !* URL : <https://zestedesavoir.com/tutoriels/3939/jouons-a-implementer-une-transformee-de-fourier-rapide/> (visité le 28/04/2026).
- [4] Mickaël PÉCHAUD. *Qu'est-ce-qu'un Dirac ?* 2018.
- [5] Jimmy ROUSSEL. *Régime sinusoïdal forcé*. URL : <https://femto-physique.fr/electrocinetique/regime-sinusoidal.php> (visité le 12/03/2026).

- [6] François SCHWARZENTRUBER. *ALGO1 - Diviser pour régner*. 2020.
- [7] Contributeurs WIKIPÉDIA. *Just-in-time compilation*. URL : https://en.wikipedia.org/wiki/Just-in-time_compilation (visité le 01/05/2026).
- [8] contributeurs WIKIPÉDIA. *Équation de la chaleur*. URL : https://fr.wikipedia.org/wiki/%C3%A9quation_de_la_chaleur (visité le 15/03/2026).
- [9] contributeurs WIKIPÉDIA. *JPEG*. URL : <https://fr.wikipedia.org/wiki/JPEG> (visité le 28/04/2026).
- [10] contributeurs WIKIPÉDIA. *Master theorem*. URL : https://fr.wikipedia.org/wiki/Master_theorem (visité le 14/04/2026).
- [11] contributeurs WIKIPÉDIA. *Traitement du signal*. URL : https://fr.wikipedia.org/wiki/Traitement_du_signal (visité le 10/03/2026).
- [12] contributeurs WIKIPÉDIA. *Transformation de Fourier discrète*. URL : https://fr.wikipedia.org/wiki/Transformation_de_Fourier_discr%C3%A8te (visité le 10/03/2026).
- [13] contributeurs WIKIPÉDIA. *Transformation de Fourier rapide*. URL : https://fr.wikipedia.org/wiki/Transformation_de_Fourier_rapide (visité le 10/03/2026).
- [14] contributeurs WIKIPÉDIA. *Transformée en cosinus discrète*. URL : https://fr.wikipedia.org/wiki/Transform%C3%A9e_en_cosinus_discr%C3%A8te (visité le 28/04/2026).